```
# Handwritten Digit Recognition using Machine Learning
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)

# ---------- 1. Load data ----------
digits = load_digits()
X, y = digits.data, digits.target
print("Images:", X.shape[0], "| Pixels per image:", X.shape[1], "| Classes:", len(np.unique(y)))
print("Image size: 8 x 8 | Pixel value range:", int(X.min()), "to", int(X.max()))

# ---------- 2. Train/test split ----------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
print("Train:", X_train.shape, "| Test:", X_test.shape)

# ---------- 3. Define models ----------
# All pixels are already on the same scale (0 to 16), so no scaling is needed.
models = {
    "Logistic Regression": LogisticRegression(max_iter=3000),
    "KNN":                 KNeighborsClassifier(n_neighbors=3),
    "SVM (RBF)":           SVC(random_state=42),
    "Decision Tree":       DecisionTreeClassifier(random_state=42),
    "Random Forest":       RandomForestClassifier(n_estimators=200, random_state=42),
    "Naive Bayes":         GaussianNB(),
}

# ---------- 4. Train + evaluate ----------
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
rows = []
for name, model in models.items():
    cv = cross_val_score(model, X_train, y_train, cv=skf, scoring="accuracy").mean()
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    rows.append({"Model": name,
                 "CV Acc": round(cv, 4),
                 "Accuracy": round(accuracy_score(y_test, pred), 4),
                 "Precision": round(precision_score(y_test, pred, average="macro"), 4),
                 "Recall": round(recall_score(y_test, pred, average="macro"), 4),
                 "F1": round(f1_score(y_test, pred, average="macro"), 4)})
```

```
results = pd.DataFrame(rows).sort_values("Accuracy", ascending=False).reset_index(drop=True)
print("\n=== MODEL COMPARISON ===")
print(results.to_string(index=False))

# ---------- 5. Best model ----------
best_name = results.loc[0, "Model"]
best = models[best_name]
best_pred = best.predict(X_test)
print("\nBest model:", best_name)
print(classification_report(y_test, best_pred, digits=3))

# ---------- 6. INNOVATION 1: PCA - fewer pixels, same accuracy? ----------
print("=== PCA: how many features do we really need? ===")
pca_rows = []
for k in [5, 10, 15, 20, 30, 40, 64]:
    m = make_pipeline(PCA(n_components=k), SVC(random_state=42))
    m.fit(X_train, y_train)
    acc = accuracy_score(y_test, m.predict(X_test))
    pca_rows.append((k, acc))
    print(f"Components = {k:>2}  ->  accuracy = {acc:.4f}")

# ---------- 7. INNOVATION 2: robustness to noisy images ----------
print("\n=== NOISE TEST: accuracy when test images are noisy ===")
rng = np.random.RandomState(0)
noise_levels = [0, 1, 2, 3, 4]
noise_res = {name: [] for name in models}
for s in noise_levels:
    X_noisy = np.clip(X_test + rng.normal(0, s, X_test.shape), 0, 16)
    for name, model in models.items():
        noise_res[name].append(accuracy_score(y_test, model.predict(X_noisy)))
noise_df = pd.DataFrame(noise_res, index=[f"noise={s}" for s in noise_levels]).round(3)
print(noise_df.T.to_string())

# ---------- 8. Plots ----------
fig, axes = plt.subplots(2, 8, figsize=(9, 2.9))
for ax, img, lab in zip(axes.ravel(), digits.images[:16], y[:16]):
    ax.imshow(img, cmap="gray_r"); ax.set_title(str(lab), fontsize=9); ax.axis("off")
plt.suptitle("Sample images from the dataset (label above each image)", fontsize=10)
plt.tight_layout(); plt.savefig("shots/samples.png", dpi=160); plt.close()

plt.figure(figsize=(5, 3.4))
pd.Series(y).value_counts().sort_index().plot(kind="bar", color="steelblue", rot=0)
plt.xlabel("Digit"); plt.ylabel("Images"); plt.title("Images per digit")
plt.tight_layout(); plt.savefig("shots/classdist.png", dpi=160); plt.close()

cm = confusion_matrix(y_test, best_pred)
fig, ax = plt.subplots(figsize=(5.6, 4.8))
ax.imshow(cm, cmap="Blues")
for i in range(10):
    for j in range(10):
        ax.text(j, i, cm[i, j], ha="center", va="center", fontsize=8,
                color="white" if cm[i, j] > cm.max() / 2 else "black")
ax.set_xticks(range(10)); ax.set_yticks(range(10))
ax.set_xlabel("Predicted digit"); ax.set_ylabel("Actual digit")
ax.set_title(f"Confusion Matrix ({best_name})")
plt.tight_layout(); plt.savefig("shots/cm.png", dpi=160); plt.close()

wrong = np.where(best_pred != y_test)[0]
```

```
n = min(len(wrong), 8)
fig, axes = plt.subplots(1, max(n, 1), figsize=(9, 2.2))
axes = np.atleast_1d(axes)
for ax, i in zip(axes, wrong[:n]):
    ax.imshow(X_test[i].reshape(8, 8), cmap="gray_r")
    ax.set_title(f"True {y_test[i]}\nPred {best_pred[i]}", fontsize=9); ax.axis("off")
plt.suptitle(f"Images the {best_name} model got wrong", fontsize=10)
plt.tight_layout(); plt.savefig("shots/wrong.png", dpi=160); plt.close()

ks, accs = zip(*pca_rows)
plt.figure(figsize=(5.5, 4))
plt.plot(ks, accs, "o-", color="darkgreen")
plt.xlabel("Number of PCA components (out of 64)"); plt.ylabel("Test accuracy")
plt.title("Accuracy with fewer features (SVM + PCA)"); plt.grid(alpha=0.3)
plt.tight_layout(); plt.savefig("shots/pca.png", dpi=160); plt.close()

plt.figure(figsize=(6, 4.2))
for name in models:
    plt.plot(noise_levels, noise_res[name], "o-", label=name)
plt.xlabel("Amount of noise added to test images"); plt.ylabel("Test accuracy")
plt.title("How well do the models handle noisy images?")
plt.legend(fontsize=7); plt.grid(alpha=0.3)
plt.tight_layout(); plt.savefig("shots/noise.png", dpi=160); plt.close()
print("\nPlots saved.")
```